AWS Cloud Operations Blog

Implementing a custom ConfigSource in Quarkus using AWS AppConfig

by Fabio Oliveira | on 15 FEB 2023 | in Advanced (300), AWS Config, Best Practices, Customer Solutions, Management & Governance, Management Tools, Technical How-to | Permalink | Share

Most systems developed on the cloud nowadays implement a microservices architecture. A common demand is that each microservice is highly configurable and that configuration can be changed without changing code, and ideally, without restarting a running service instance. Quarkus (see https://quarkus.io/) is a popular framework for writing high-performing microservices in Java. AWS AppConfig is AWS' service for continuous configuration, a concept well described by Werner Vogels. In this blog post you will learn how to programmatically integrate continuous configuration, feature flags into a Quarkus-based microservice framework using the standard extension mechanism. This enables Quarkus developers to leverage all of AWS AppConfig features and architectural benefits using standard Quarkus coding idioms.

To illustrate dynamic reconfiguration, we employ a fictional use case calculating banking loans for marketing purposes, e.g. to be used in online calculators for non-binding offers. The requirement of the fictional bank is being able to change the interest rate used for calculation multiple times a day. The bank does not need to change the code, update the deployment or restart the service instances each time this happens. Instead, the code has the interest rate in its dynamic configuration data so it can be updated without a code deployment. We will implement a solution to the fictional challenge of the bank by integrating Quarkus with AWS AppConfig.

To explain how we can achieve this integration, we will firstly explore how configuration works in Quarkus by looking at the MicroProfile Config standard. Afterwards we will set up sample AWS AppConfig resources to create a test environment. Lastly, we implement a custom Config source to access AWS AppConfig and make obtained configuration available to application code via the standard @ConfigProperty annotation as usual in Quarkus.

The sample code to illustrate this blog post is available at https://github.com/aws-samples/aws-appconfig-with-quarkus/ and will be heavily cited.

How MicroProfile Config in Quarkus works

Before we start integration, we need to understand a little bit about how configuration works in Quarkus. What is a ConfigSource and why do we need it anyway? Well, Quarkus implements a standard called MicroProfile Config, which is part of the wider set of MicroProfile specifications. The purpose of this specification is to enable an easy and extensible configuration mechanism for services. Within the application the configuration can conveniently be accessed by annotating member variables of CDI beans with @ConfigProperty.

A service can have multiple ConfigSources from which it can obtain values for config properties. ConfigSources are ordered by precedence to determine which config value will be injected if two or more ConfigSources have a value of a config property set. Standard ConfigSources enable reading from environment variables or Java System properties. Programmers can implement and register their own ConfigSources to extend the system, which we will do to read configuration from AWS AppConfig as an additional source.

Figure 1 illustrates how ConfigSources take precedence over one another based on their assigned ordinal number. To determine the value for config value A and config value B one would check the ConfigSources in descending ordinal number for the first which defines a value for the config property and resolve to this value.

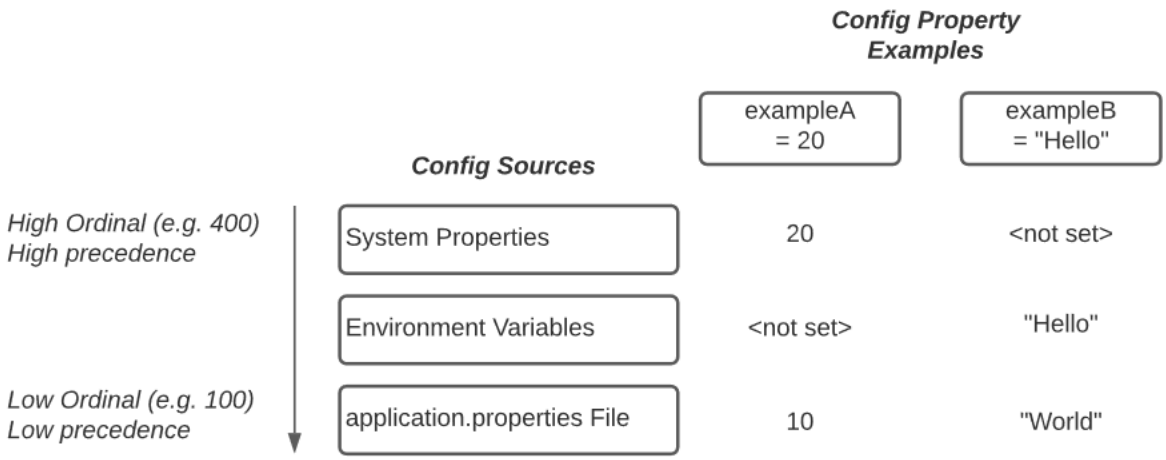

Figure 1. Illustration of ConfigSource Precedence with two examples of value resolution for a config value

When it is time to implement our ConfigSource for AWS AppConfig later in this blog post, we will pick an ordinal number which positions the custom ConfigSource at a sensible precedence. This avoids issues with common development techniques like overriding config properties during testing using easy to access, high precedence config sources like environment variables. Quarkus uses ordinal 300 for environment variables and 250 for application.properties. Choosing 275 as the ordinal value of our ConfigSource will create a precedence as illustrated in figure 2, which we will choose for this blog post.

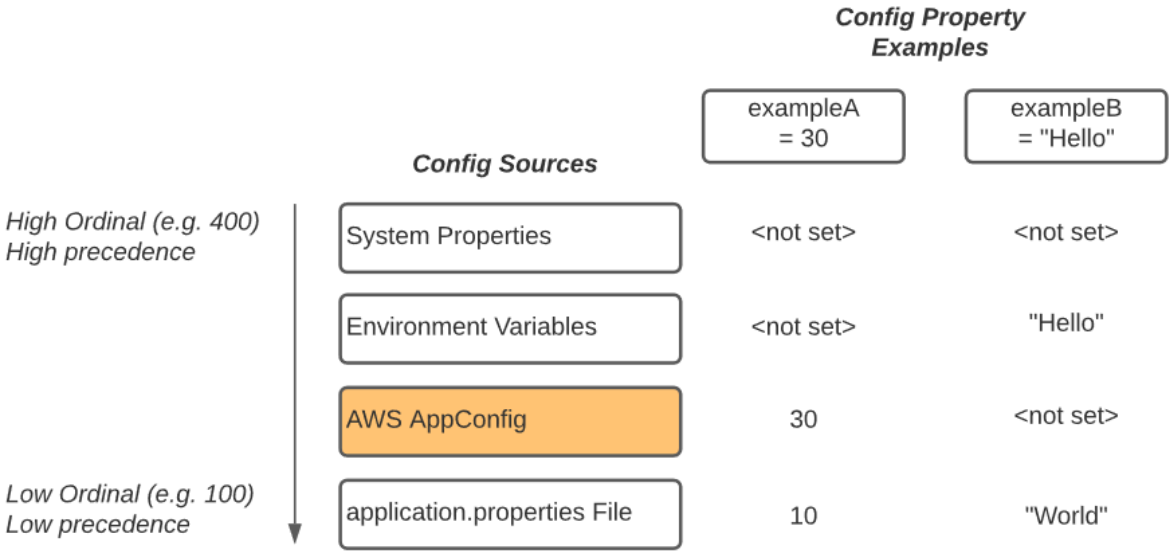

Figure 2. Target precedence of ConfigSources with ordinal 275 applied to "AWS AppConfig" ConfigSource

Setting up AWS AppConfig for integration

Next, we create a test environment in AWS AppConfig. The setup consists of an Application, an Environment, and a Configuration Profile (in this case, we will use HostedConfigurationStore). It also defines a custom

DeploymentStrategy with a Step Percentage of 100% and an interval of 1 minute, so we can easily experiment in our test environment and do not have to wait too long to observe the effects of rolling out new configuration. It should be noted that slower deployment times (in minutes, days, or weeks) are a recommended best practice in order to limit the blast radius of any changes. For this testing, however, we have a short deployment and bake time. Figure 3 illustrates the setup. An CDK app for easy setup is included in the sample repository referenced at the beginning of the blog post.

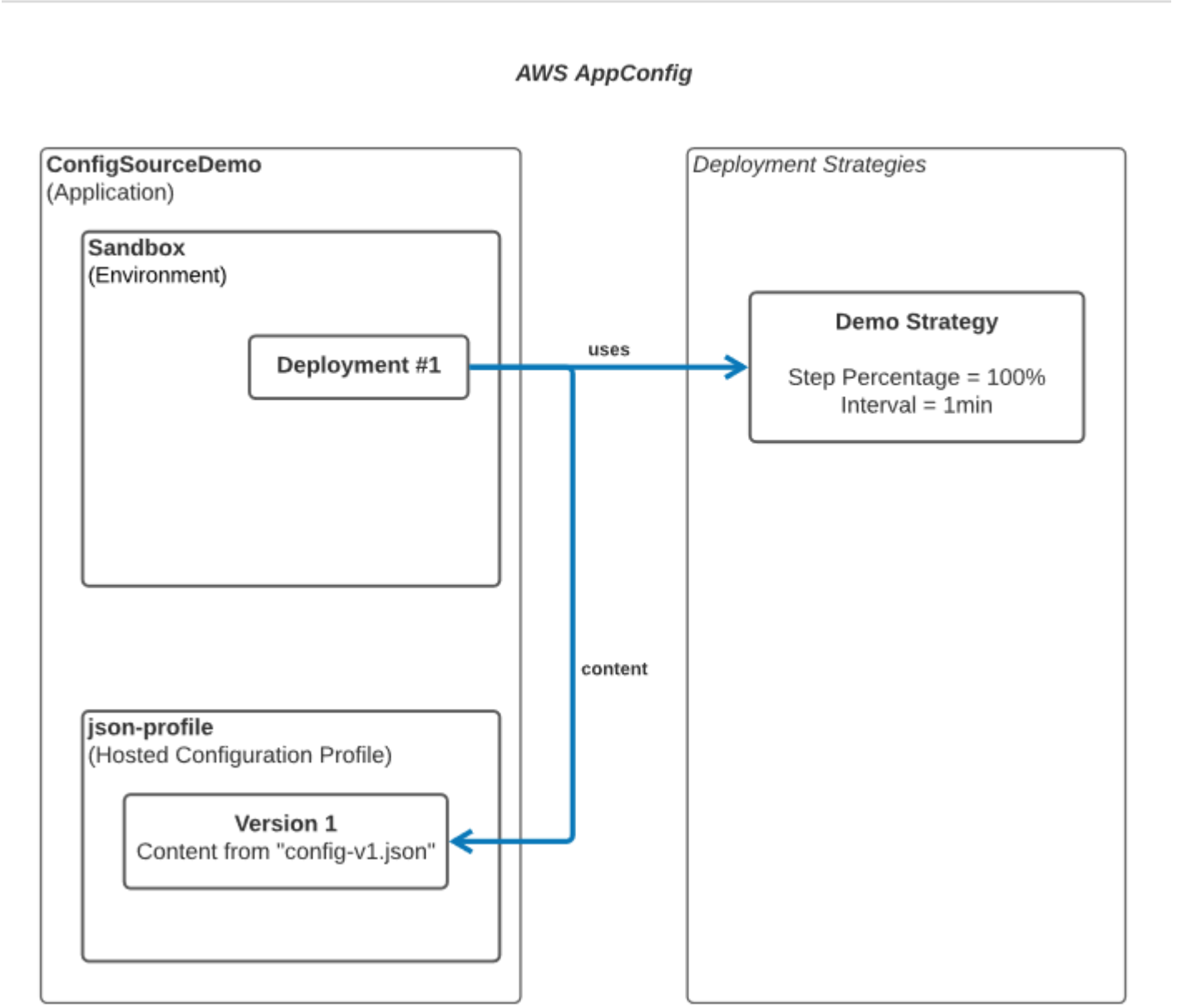

Figure 3. Demo setup in AWS AppConfig created by sample AWS CDK app

Implementing the AWS AppConfig ConfigSource

With our understanding of MicroProfile Config and our AWS AppConfig test environment we can start implementing the actual AWS AppConfig/Quarkus integration by implementing a custom ConfigSource. The sample implementation is included in the repository linked at the beginning of the blog post.

A custom ConfigSource must implement the interface org.eclipse.microprofile.config.spi.ConfigSource and afterwards be registered so Quarkus can pick it up during bootstrapping at application startup. To register the custom ConfigSource, create a file /META-INF/services/org.eclipse.microprofile.config.spi.ConfigSource and a

the fully qualified class name. Note that this class is not part of the standard CDI container and thus dependency injection will not work in the class!

Our ConfigSource needs the correct level of precedence, which we set to 275 as explained earlier. Quarkus registers its own property files (i.e., application.properties) with ordinal 250 and we want to be able to override those values with config provisioned through AWS AppConfig, but we do not want to override environment variables or system properties from AWS AppConfig.

```
public class AwsAppConfigSource implements ConfigSource {

  private static final int ORDINAL = 275;

  // code omitted for legibility

  @Override
  public int getOrdinal() {
    return ORDINAL;
  }

}
```

Our implementation is for demonstration purposes and thus will be straight forward and simplified; you may want to customize this to your specific needs in case you adopt it. The constructor will create a synchronous AWS AppConfig client and schedule a method to poll for new configuration every 30 seconds.

```
public class AwsAppConfigSource implements ConfigSource {

  private String token = null;

  public AwsAppConfigSource() {
    client = AppConfigDataClient.builder().build();
    props = new ConcurrentHashMap();

    execSvc = new ScheduledThreadPoolExecutor(1);

    token = startConfigurationSession();

    fetchConfig();
    LOGGER.info("initialized with AppConfig provided interestRate = {}", props.get('

    // we poll in 30 seconds intervals for demonstration purposes
    execSvc.scheduleAtFixedRate(this::fetchConfig, 30, 30, TimeUnit.SECONDS);
  }

  private String startConfigurationSession() {
    // explained later
```

```
  }

  private void fetchConfig() {
     // explained later
  }

  // code omitted for legibility

}
```

The method fetching the configuration from AWS AppConfig must take care of two issues:

Firstly, we need to track the configuration token of the last delivered configuration of AWS AppConfig. This is required to avoid unnecessary network traffic of redelivering the configuration all the time, and it would also incur extra charges for the AWS AppConfig service.

Secondly, we want to make a thread-safe and atomic update to our stored configuration to ensure Quarkus injects mutually consistent values of a single configuration version. This issue applies to all dynamic configuration mechanisms and is not specific to AWS AppConfig. In this blog post we will not dive deeper into this issue.

```
public class AwsAppConfigSource implements ConfigSource {

  private ConcurrentHashMap<String, String> props;

  private AppConfigDataClient dataClient;
  private String token;

  private final ObjectMapper mapper;
  private final TypeReference<ConcurrentHashMap<String, String>> typeRef;

  private String startConfigurationSession() {
    var applicationName = "ConfigSourceDemo";
    var environmentName = "Sandbox";
    var configProfileName = "json-profile";

    var res =
        dataClient.startConfigurationSession(
            req -> {
              req.applicationIdentifier(applicationName);
              req.configurationProfileIdentifier(environmentName);
              req.environmentIdentifier(configProfileName);
            });

    return res.initialConfigurationToken();
  }
```

```
  private void fetchConfig() {
    var res =
        dataClient.getLatestConfiguration(
            req -> {
              req.configurationToken(token);
            });

    try {
      if (res.configuration() == null) {
        LOGGER.info("unexpected null value for configuration. aborting config update
        return;
      }

      var configString = res.configuration().asUtf8String();
      if(configString.isEmpty()) {
        LOGGER.info("No changes on the configuration received");
        return;
      }

      props = mapper.readValue(configString, typeRef);

      token = res.nextPollConfigurationToken();

      LOGGER.debug("now using config version = {}\nand interestRate = {}", token, pr
    } catch (Exception ex) {
      LOGGER.error("unexpected failure in reading the config", ex);
      return;
    }
  }

  // code omitted for legibility

}
```

To have our ConfigSource offer configuration values fetched from AWS AppConfig we need to override a couple of methods.

```
public class AwsAppConfigSource implements ConfigSource {

  // code omitted for legibility

  @Override
  public Map<String, String> getProperties() {
    return props;
  }

  @Override
```

```
    public Set<String> getPropertyNames() {
      return props.keySet();
    }

    @Override
    public String getValue(String s) {
      return props.get(s);
    }

    @Override
    public String getName() {
      return "AwsAppConfigSource";
    }
  }
```

Putting it all together

By now we have learned how MicroProfile Config works, we set up a test environment in AWS AppConfig to use with our exemplary ConfigSource implementation and we implemented the custom ConfigSource to vend AWS AppConfig configuration data directly into Quarkus. Let's check if it works!

We again leverage the sample code linked at the beginning of the blog post. All testing demonstrated here will use your locally configured AWS credentials, config and selected profile, so be sure to point to an appropriate account and region for testing purposes. Make sure that you set the AWS_PROFILE environment variable in all shells from which you run the sample code.

If you have not done so yet, provision the sample CDK app from cdk/ folder in the sample code repository. Next, we build and run the sample application. We will not use the quarkus:dev maven goal as it uses a second classloader, which will load and run our ConfigSource, too. This would duplicate log entries where you just expect single log entries.

```
# in repository root directory
./mvnw clean package
java -jar target/quarkus-app/quarkus-run.jar
```

The sample application uses the default credential chain to fetch the configuration from AWS AppConfig as provisioned earlier using the CDK app. Notice that we, for demonstration purposes, hardcoded those values in the source code of AwsAppConfigSource class! In a production scenario you may want to read these values directly from the environment to bootstrap your application configuration mechanism. Remember, we bootstrap the configuration framework, so we cannot use the configuration yet 😉 .

Having the application running for two minutes, you should see a log output as shown below. Our sample implementation queries in the specified interval AWS AppConfig for new configuration versions.

```
__  ____  __  _____   ___  __ ____  ______
 --/ __ \/ / / / _ | / _ \/ //_/ / / / __/
 -/ /_/ / /_/ / __ |/ , _/ ,< / /_/ /\ \
--\___\_\____/_/ |_/_/|_/_/|_|\____/___/
2021-11-10 11:38:16,165 DEBUG [io.eic.con.sou.AwsAppConfigSource] (main) now using c
2021-11-10 11:38:16,165 INFO  [io.eic.con.sou.AwsAppConfigSource] (main) initialized
2021-11-10 11:38:16,321 INFO  [io.quarkus] (main) appconfig-quarkus 1.0-SNAPSHOT on
2021-11-10 11:38:16,322 INFO  [io.quarkus] (main) Profile prod activated.
2021-11-10 11:38:16,322 INFO  [io.quarkus] (main) Installed features: [cdi, resteasy
2021-11-10 11:38:46,214 DEBUG [io.eic.con.sou.AwsAppConfigSource] (pool-4-thread-1)
2021-11-10 11:39:16,213 DEBUG [io.eic.con.sou.AwsAppConfigSource] (pool-4-thread-1)
2021-11-10 11:39:46,197 DEBUG [io.eic.con.sou.AwsAppConfigSource] (pool-4-thread-1)
2021-11-10 11:40:16,223 DEBUG [io.eic.con.sou.AwsAppConfigSource] (pool-4-thread-1)
```

You can observe that the application fetched an interest rate of 15% from AWS AppConfig. This value is distinct from the default value in the LoanResource class and is exactly the value which was provisioned using the CDK app and the file config-v1.json in the cdk/ folder.

Let's open a shell and execute the sample API using the command below:

```
curl localhost:8080/loan/annuity -d '{"amountInEur":1000, "durationInYears":1}' -H '
# Response: {"annuity":1150.0000000000007}
```

We finally get to the magic moment which we have been building towards for so long: Dynamically reconfiguring our service without redeployments and restarts, using the power of AWS AppConfig!

We trigger the AWS AppConfig Deployment with an updated configuration version by a simple modification in the CDK app. Of course, you could use the AWS AppConfig API Actions directly, or any other mechanism which suits your scenario.

In the file cdk/lib/app-config-stack.ts change line 36 from

```
// Change this line and run 'cdk deploy' to update the config
content: initialConfigContent,
```

to

```
// Change this line and run 'cdk deploy' to update the config
content: updatedConfigContent,
```

You can preview the changes comfortably by running

```
cdk diff
```

which should provide an output similar to the below:

```
Stack AppConfigStack
Resources
[~] AWS::AppConfig::HostedConfigurationVersion ac-hcv-v1 achcvv1 replace
 └─ [~] Content (requires replacement)
     ├─ [-] {
  "interestRate": 0.15
}
     └─ [+] {
  "interestRate": 0.01
}
[~] AWS::AppConfig::Deployment ac-dep-alpha acdepalpha replace
 └─ [~] ConfigurationVersion (requires replacement)
     └─ [~] .Ref:
         ├─ [-] achcvv1
         └─ [+] achcvv1 (replaced)
```

Execute the update by running the command listed below.

```
cdk deploy
```

This CDK command will take about 1 minute to complete, as it immediately triggers a new AWS AppConfig deployment to our environment and awaits completion of this configuration rollout. This is due to the integrated stack defined in the CDK app.

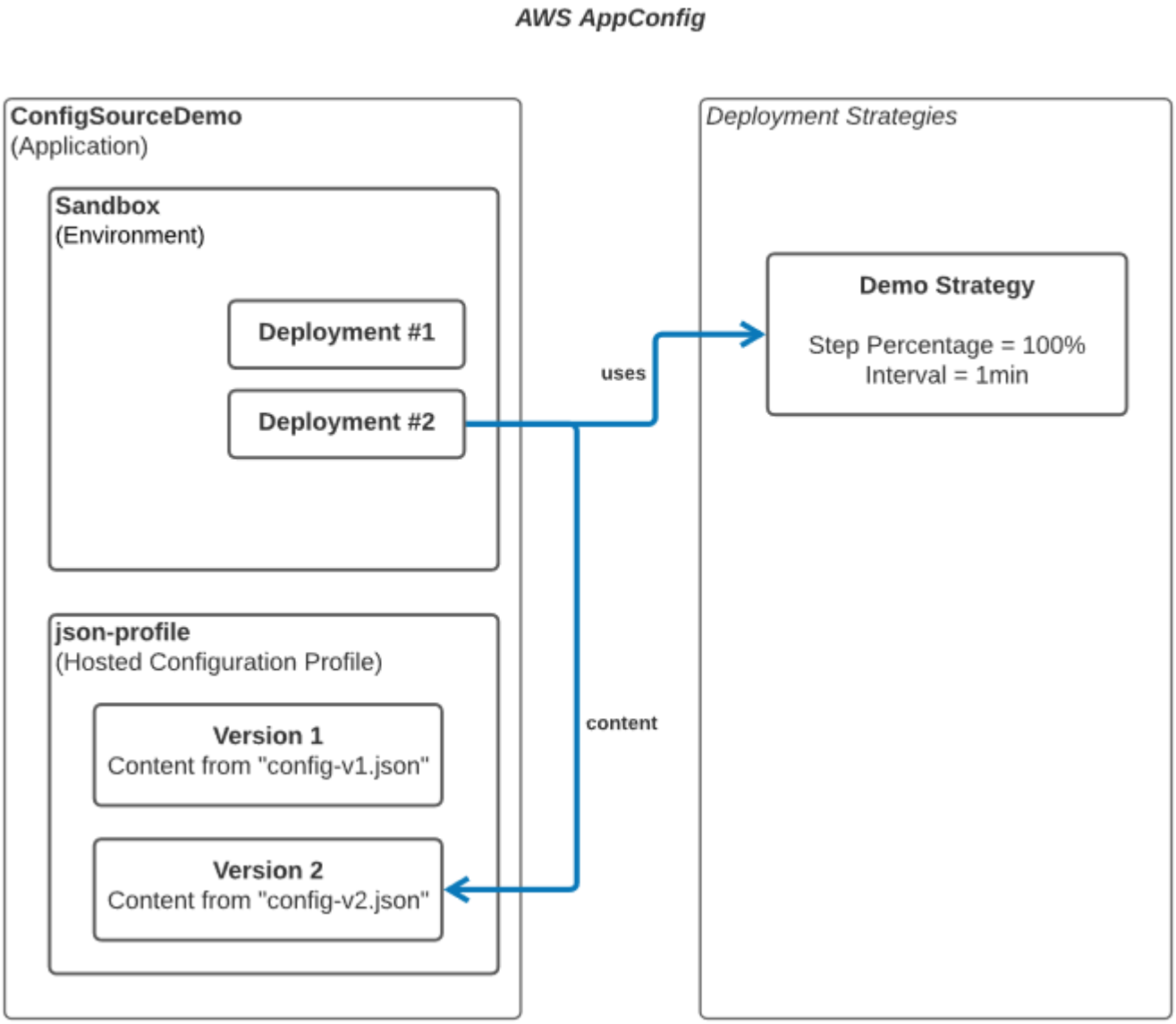

Figure 4. illustrates our new AWS AppConfig resources.

At some point during the CDK deployment, and hence the rollout of the new configuration version including the lowered interest rate value, you will observe in the logs that our running application picked up the new config version:

```
2021-11-10 11:58:16,197 DEBUG [io.eic.con.sou.AwsAppConfigSource] (pool-4-thread-1)
2021-11-10 11:58:46,210 DEBUG [io.eic.con.sou.AwsAppConfigSource] (pool-4-thread-1)
2021-11-10 11:59:16,203 DEBUG [io.eic.con.sou.AwsAppConfigSource] (pool-4-thread-1)
2021-11-10 11:59:46,213 DEBUG [io.eic.con.sou.AwsAppConfigSource] (pool-4-thread-1)
2021-11-10 12:00:16,197 DEBUG [io.eic.con.sou.AwsAppConfigSource] (pool-4-thread-1)
```

If you test the API now, you will see that it is using the new config property value — we achieved our goal of dynamic reconfiguration!

```
curl localhost:8080/loan/annuity -d '{"amountInEur":1000, "durationInYears":1}'
```

```
# Response: {"annuity":1009.9999999999991}
```

Before getting to the summary, I would like to point out an important, subtle detail in the implementation of the sample service. When you inspect the class LoanResource in the sample code, you will see that the configuration value is injected to a member variable of type Provider<Double> instead of a plain double.

```
@ConfigProperty(name = "interestRate", defaultValue = "0.05")
Provider<Double> interestRate;
```

The reason for this is that Quarkus uses a Singleton CDI scope for JAX-RS resource classes[1], and configuration injection happens only at instantiation time. Wrapping with javax.inject.Provider allows to dynamically fetch the respective config value at any time without the need to instantiate a new instance of the class.

Besides this little complication inherit to the CDI/JAX-RS lifecycle management the code above is perfectly decoupled from AWS AppConfig. The application developer works with his familiar abstractions but is served configuration directly from AWS AppConfig.

Summary

In this blog post we learned quite a lot! We understood how the configuration mechanism in Quarkus can be extended and how we can access configuration stored in AWS AppConfig over the standard @ConfigProperty API available in Quarkus, so developers can enjoy the smooth developer experience they are used to in Quarkus. We also saw how beneficial and easy it is to roll out new configuration via AWS AppConfig without application restarts instead of changing the packaged application.properties file of Quarkus.

I hope you enjoyed reading this blog post and got a good idea on how to leverage AWS AppConfig within your Quarkus microservice for improved configurability!

Cleaning up

Make sure to tear down the CDK app by running:

```
# in folder cdk/
cdk destroy
```

About the authors:

TAGS: AWS AppConfig, AWS CDK, Java, Quarkus